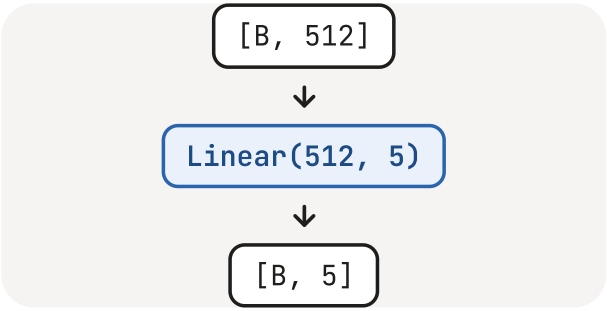
Transfer learning retrieval

Recall 1 min · Discuss 2 min

1. What is the backbone?
2. What changes for new target labels?
3. Which parameters train with frozen features?

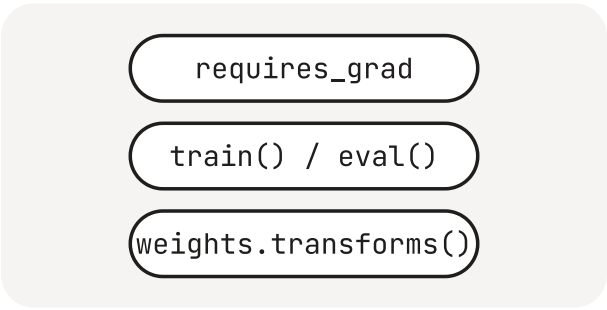
Lecture 2 learning outcomes

1



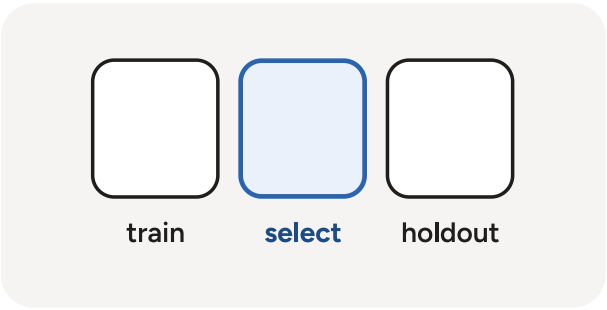
Replace and count a target classifier

2



Control trainable layers and input processing

3



Select models using validation evidence

Adaptation workflow

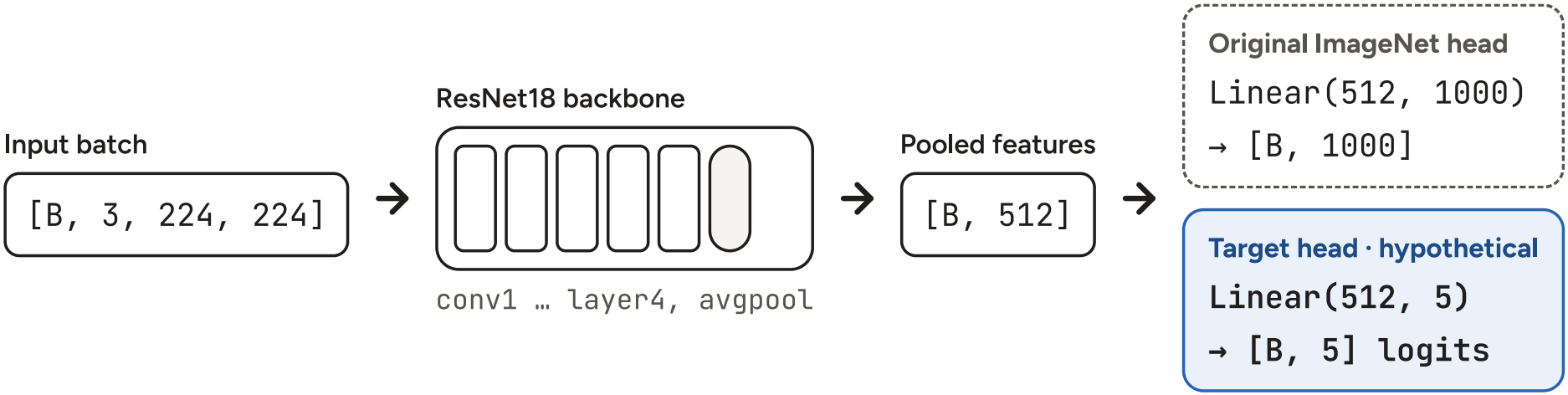
Define target labels and independent splits

Load weights, replace the head, and train

Select on validation, then evaluate the holdout

1. Split independent originals: train, validation, holdout
2. Load a named pretrained weight set
3. Apply the weight set's preprocessing
4. Replace the head for five classes
5. Choose trainable layers and train
6. Select settings and checkpoint on validation
7. Evaluate the selected model on the holdout

ResNet18 target classifier shapes

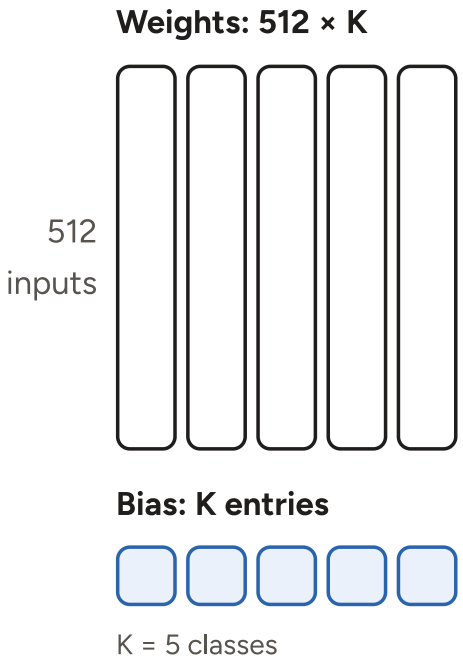


Target indices: 0 apple · 1 banana · 2 orange · 3 pear · 4 grape

- The pooled representation has 512 features
- The original head predicts 1000 classes
- The target head predicts five classes

Classifier parameter calculation

- A linear head has 512K + K parameters
- Five classes require 2,565 parameters
- Only the new head trains in the frozen setup



Five classes

$$512 \times 5 + 5$$

$$= 2{,}565$$

How many parameters for four classes?

Classifier replacement in PyTorch

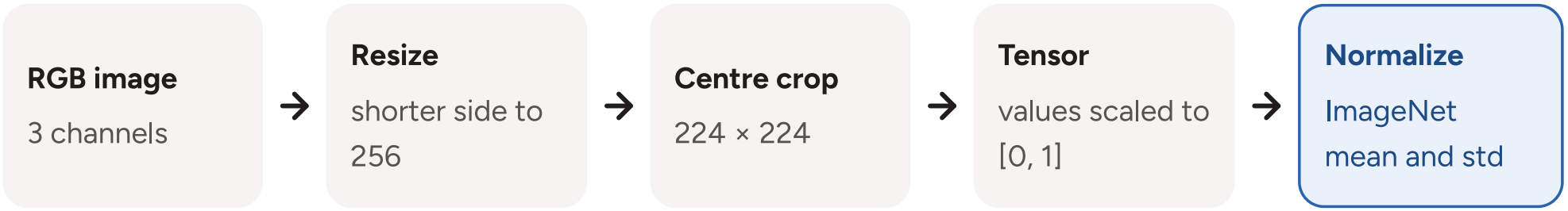
1. Load an explicit pretrained weight set
2. Freeze the backbone parameters
3. Create a new five-output linear head

```
1  weights = ResNet18_Weights.IMAGENET1K_V1
   model = resnet18(weights=weights)
2  for p in model.parameters():
       p.requires_grad = False
3  model.fc = nn.Linear(model.fc.in_features, 5)   → [B, 5]
   optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
```

Replacing fc after the freeze loop creates a new trainable head: its parameters default to requires_grad = True

Weight-specific input processing

Evaluation pipeline for `ResNet18_Weights.IMAGENET1K_V1`



Code `eval_transform = weights.transforms()`

- Use the named weight set's evaluation transforms
- Training augmentation must preserve labels
- Verify channels, scale, and normalization

Gradients and model modes

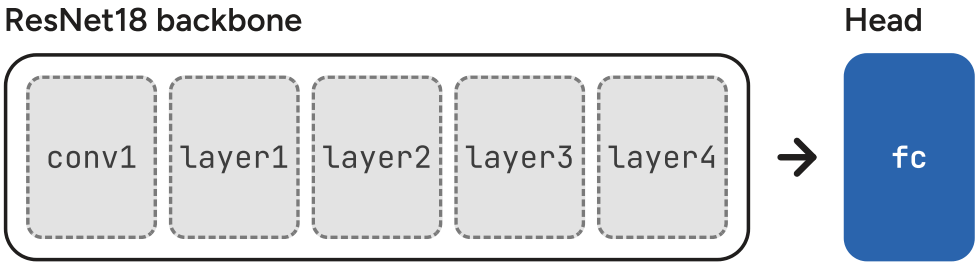
Control	What it controls	Example
`requires_grad`	controls parameter gradients	`p.requires_grad = False`
`train / eval`	control mode-sensitive layers such as BatchNorm and dropout	`model.eval()`
`no_grad`	controls gradient recording	`with torch.no_grad():` `  logits = model(x)`

Note: eval() does not disable gradients

Frozen backbone training policy



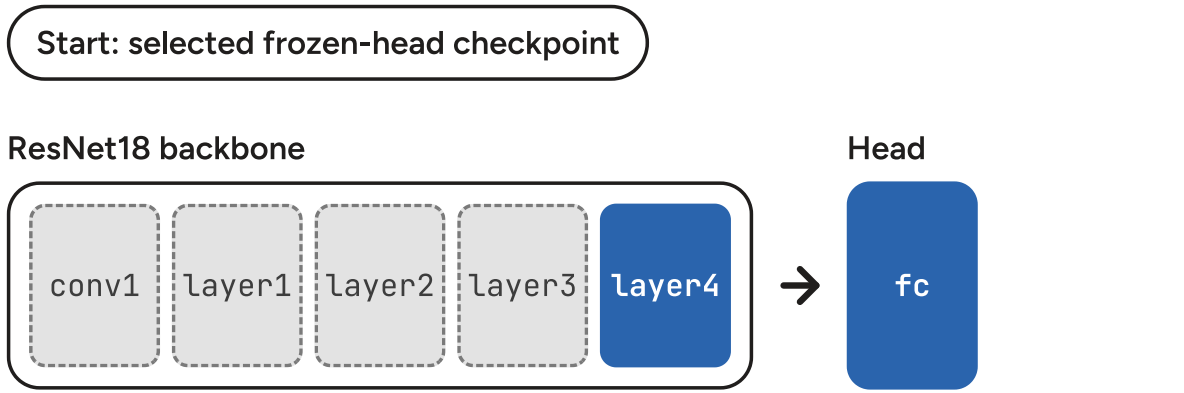
- Keep the pretrained backbone in evaluation mode
- Train the new linear head
- Check that the intended backbone state stays fixed




```
model.eval()      # fixed BatchNorm statistics
model.fc.train()  # new head in training mode
```

Selective fine-tuning

- Start from the trained frozen-head model
- Unfreeze the final residual stage (layer4) and head
- Compare the extension on validation




```
for p in model.layer4.parameters():
    p.requires_grad = True
model.eval()      # BatchNorm statistics stay fixed
model.fc.train()
```

eval() fixes BatchNorm statistics; layer4 and fc still receive gradients

Fine-tuning learning rates and optimizer

- Use a smaller learning rate for pretrained layers
- Include newly trainable parameters
- Treat starting rates as validation choices

Group 1 · pretrained stage

`model.layer4`

`lr = 0.00001`

Group 2 · new head

`model.fc`

`lr = 0.0001`

Example starting values · BatchNorm stays in eval mode

Before: head-only optimizer cannot update layer4

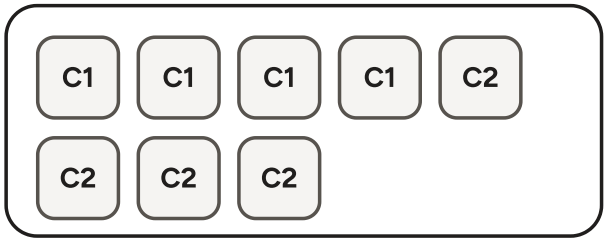
```
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
```

After: rebuilt with both parameter groups

```
optimizer = torch.optim.Adam([
    {"params": model.layer4.parameters(), "lr": 1e-5},
    {"params": model.fc.parameters(), "lr": 1e-4},
])
```

Training validation and final evaluation

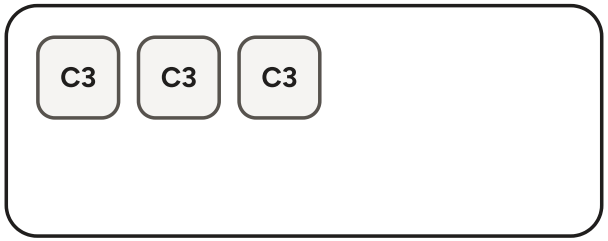
Training



+ augmentation: crops, flips

Training updates parameters

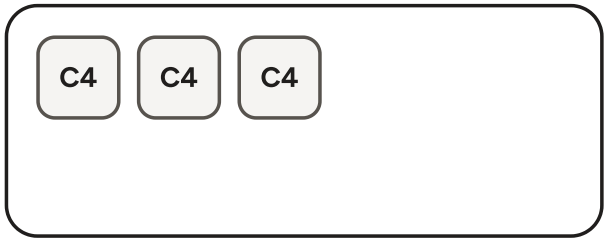
Validation



deterministic transforms

Validation chooses settings and checkpoints

Holdout



deterministic transforms

The holdout evaluates the selected procedure

Grouping example: C1–C4 are cameras; each camera's images stay in one split

Comparing adaptation experiments



- Use the same split and reporting protocol
- Report validation metrics and trainable parameters
- State hardware and the training budget

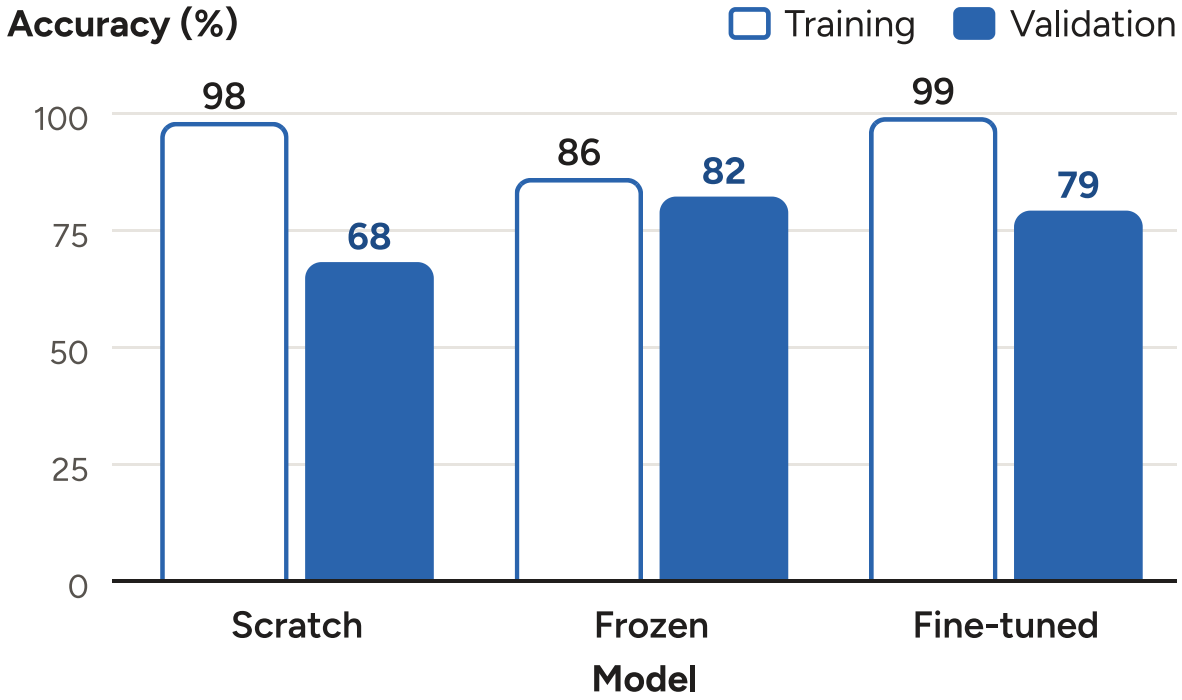
Experiment	Trainable	Report
Scratch 	All layers, random init	val accuracy, val macro F1, cost
Frozen 	fc only: 2,565	val accuracy, val macro F1, cost
Selective fine-tuning 	layer4 + fc	val accuracy, val macro F1, cost

Stated rule: select by validation macro F1; accuracy breaks ties

Interpreting validation results

- Scratch: 98% train, 68% validation accuracy
- Frozen: 86% train, 82% validation accuracy
- Fine-tuned: 99% train, 79% validation accuracy

Using validation accuracy only, which model would you select?



Illustrative values invented for discussion, same hypothetical validation split

Common implementation mistakes

✕ Incorrect	✓ Corrected
Keep the original 1000-class head	**The target task needs the correct output head**
Apply softmax before CrossEntropyLoss	**CrossEntropyLoss takes raw logits**
Unfreeze layer4 but keep the head-only optimizer	**Unfrozen layers need optimizer membership**

Domain shift and negative transfer

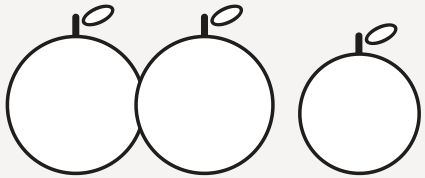
- Sensors, viewpoints, and lighting can change inputs
- Source features may miss target distinctions
- Target validation must test transfer benefit

Definition

Negative transfer occurs when transferring learned knowledge reduces target performance relative to a relevant baseline.

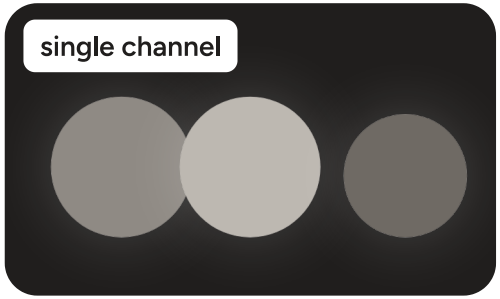
Illustrative schematics

Ordinary RGB photo



Similar to source pretraining images

Different sensor



Changed image formation and contrast

Experiment repair activity

Flawed experiment Pairs · Work 90 s · Discuss 90 s

Propose three repairs

Flaw 1

Augmented copies appear across splits

Repair

Flaw 2

The highest training accuracy wins

Repair

Flaw 3

Test scores repeatedly change the learning rate

Repair

Exit ticket and lecture recap

Write 2 min · Review 1 min

1. Calculate a four-class head with 512 features

2. Explain why freezing alone does not fix BatchNorm behavior

3. Name the data used for model selection